High Energy Particle Storms Simulation

Parallel Implementations Using MPI OpenMP and CUDA

Alessandro Crocetta | Student ID 2120774
Emanuele Trinca | Student ID 2108281

Abstract: We present two parallel versions of the high-energy particle storm simulator, one using MPI and OpenMP and the other using CUDA. In both versions, workers update different cells while keeping the original order of particle contributions. We compare CPU speedup, efficiency and scalability, and use GPU profiling to examine the CUDA kernels. Tests cover three workloads, eleven CPU configurations and block sizes from 64 to 1024 threads. All measured outputs match the sequential reference.

1 Sequential Code Analysis

The sequential core in energy_storms_core.c processes each storm in three stages [1]. First, it accumulates the particles' energy across the layer. It then averages each interior cell with its neighbours and searches for the highest positive strict local maximum. With N cells and P particles, accumulation costs O(NP), while relaxation and peak search each cost O(N). For S comparable storms, the total cost is O(SNP).

For particle j and cell k, the contribution follows the multiplication and division order in the supplied code:

$$c_j(k) = \frac{\frac{1000\ e_j}{N}}{\sqrt{|x_j - k| + 1}}$$

Here e_j and x_j are the input energy and position. Contributions below 0.001/N in absolute value are discarded. The first and last cells are never relaxed or considered as peaks. Equal positive maxima select the smaller position; without a positive peak the result is (0, 0). Successive storms depend on the layer left by the previous storm.

Accumulation is the main cost when a storm contains many particles: each particle-cell interaction requires a square root and a division. The sequential code also copies the whole layer before relaxation. This extra pass matters more for a large layer with few particles. Parallelizing the particle loop would let several workers write to the same cells, so we assign cells to workers instead.

2 Hybrid OpenMP and MPI Implementation

We split the layer into contiguous MPI domains and use OpenMP threads within each domain. Every rank has the same storm input, but stores only its own cells and two halo cells. The changes are cell ownership, pre-scaled particle energies and buffer swapping during relaxation.

2.1 Initialization and Setup

For R ranks, each rank receives floor(N/R) cells and the first N mod R ranks receive one additional cell. Global offsets preserve contiguous coverage. Owned indices are 1 through local_size, with halos at 0 and local_size+1. MPI_PROC_NULL represents missing neighbours; ranks with no cells have no active neighbours. The last nonempty rank has no right neighbour, including when R exceeds N.

We start with two zero-initialized float buffers and scale the particle energies once per storm into an auxiliary array. All MPI calls run on the main thread, outside the OpenMP regions, so the supplied MPI_Init setup does not need concurrent MPI calls. The accumulation loop uses static scheduling. Each cell visits the full particle list, including contributions that fall below the threshold, so the work per cell is similar.

2.2 The Parallel Implementation

2.2.1 The Bombardment Phase

We reverse the sequential loop order: the outer loop runs over cells in parallel, and the inner loop visits particles in their original order. A worker keeps the cell value in a local variable, adds the accepted contributions and writes the result once. This avoids atomic updates and repeated layer accesses inside the particle loop. Pre-scaling also removes a repeated division by N, although each interaction still needs the square root and attenuation division.

```
float value = layer[local_index];
for (int j = 0; j < storm_size; j++) {
    int distance = abs(pos[j] - global_index);
    float c = particle_energy[j] /
              sqrtf((float)(distance + 1));
    if (c >= threshold || c <= -threshold)
        value += c;
}
layer[local_index] = value;
```

Listing 1. Cell-owned accumulation excerpt; position access abbreviated.

Ranks do not communicate during accumulation. OpenMP setup, scheduling and synchronization still take time, and workers share the memory subsystem. Keeping the particle order avoids changes to the sum caused by floating-point reassociation.

2.2.2 The Relaxation Phase

Before relaxation, two complementary MPI_Sendrecv calls exchange the accumulated values at the domain boundaries. The halo values let each rank apply the three-point average to its edge cells. We write the interior results to the other buffer, preserve the physical endpoints and swap the buffer pointers. This removes the full-layer copy in the sequential version. The exchanges are blocking; we do not overlap them with the interior computation.

```
MPI_Sendrecv(&layer[1], 1, MPI_FLOAT,
    left, 0, &layer[local_size + 1], 1,
    MPI_FLOAT, right, 0, MPI_COMM_WORLD,
    MPI_STATUS_IGNORE);
/* Complementary exchange of last owned cell. */
```

Listing 2. Excerpt showing the first of the two boundary exchanges.

2.2.3 Peak Search and Global Reduction

After relaxation, we exchange the halos again. The earlier values are no longer valid for peak search and could give the wrong result at a partition boundary. Each OpenMP thread searches its own cells and keeps a private value-position pair. A short critical section merges these candidates within the rank. Values are compared using the MPI_FLOAT_INT struct, while MPI_MAXLOC identifies the maximum value and its corresponding rank. Finally MPI_Reduce returns the global result to rank 0 [2].

Every merge prefers greater energy and then the smaller global position. The neutral candidate is (0.0f, 0), excluding zero and negative peaks just as in the baseline. Unlike an all-gather, the implemented collective returns the winning pair only to the rank that prints it. Each storm therefore executes accumulation, halo exchange, relaxation, buffer swap, halo exchange, local search and global reduction.

2.3 OpenMP and MPI Performance Analysis

We ran all timing tests on the same i7-13700H laptop under WSL2. Each configuration has one discarded warm-up and five measured runs; ratios use the mean. We tested R = 1, 2, 4 ranks and T = 1, 2, 4, 8 threads per rank. We excluded R=4, T=8 because this would exceed the 20 logical CPUs. No explicit CPU affinity was set. Appendices A and B give the hardware, inputs and compilation flags.

2.3.1 Absolute Speedup Analysis

We define absolute speedup as S(R,T) = T_seq / T_hybrid(R,T). Both executables are compiled with -O3, so the baseline already includes compiler optimization. In Figure 1, colours identify the workloads and line styles and markers identify the MPI rank counts.

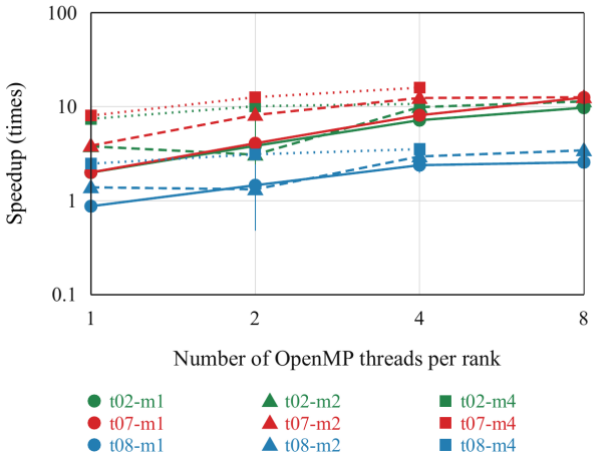


Figure 1. Absolute speedup over the optimized sequential baseline. m1, m2 and m4 denote MPI ranks. Whiskers use the parallel minimum and maximum with the sequential mean held fixed. Four-rank curves stop at four threads per rank.

For test_02, the fastest hybrid configuration is 2 ranks x 8 threads (11.39x).

For test_07 and test_08, it is 4 ranks x 4 threads, with speedups of 15.94x and 3.54x.

Single-worker performance varies by workload; test_08 is slower than the sequential baseline in Figure 1. Absolute speedup includes implementation changes and the effect of additional workers.

2.3.2 Parallel Efficiency Analysis

Efficiency is E(R,T) = S(R,T)/(RT), the speedup per CPU worker relative to the sequential baseline. Values above one are possible here because the hybrid code changes the sequential loop organization, even with a single worker and the same optimization level.

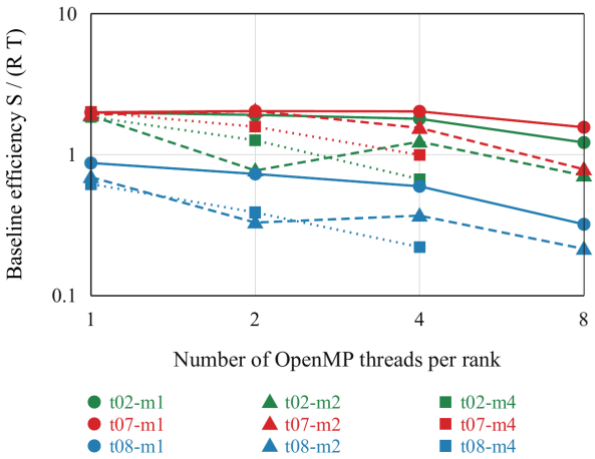


Figure 2. Baseline-relative efficiency S/(R T), with a logarithmic vertical axis. Values above one reflect implementation gains as well as parallel execution.

For the same total worker count, using more ranks means fewer OpenMP threads per rank and more MPI synchronization. The memory-heavy workload has less arithmetic per cell to offset setup and communication costs.

2.3.3 Parallel Scalability Analysis

To separate the single-worker code changes from parallel scaling, we use G(R,T) = T_hybrid(1,1)/T_hybrid(R,T). The dashed line in Figure 3 shows ideal T scaling for one MPI rank. For the multi-rank curves, the total number of workers is R T, so comparisons must account for both ranks and threads.

Amdahl's Law models fixed-input scaling with p workers and serial fraction f [5]:

$$S(p) = \frac{1}{f + \frac{1 - f}{p}}$$

The dashed ideal line corresponds to f=0 with no parallel overhead.

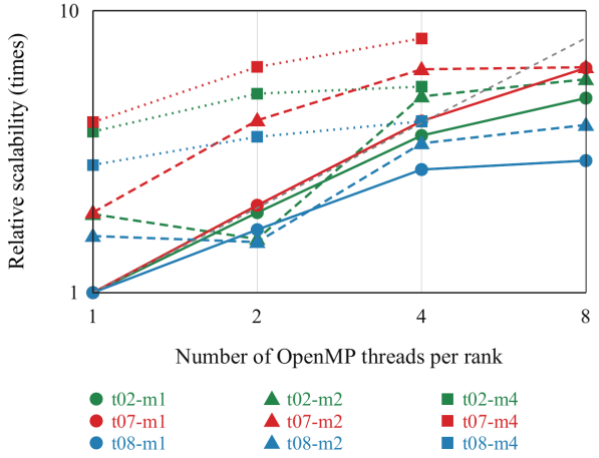


Figure 3. Relative scalability against hybrid 1 x 1. The dashed reference is ideal one-rank scaling; CPU worker count is R T.

With one rank and eight threads, we measure 4.90x scalability for test_02, 6.27x for test_07 and 2.94x for test_08, compared with the ideal 8x. Serial work is only one possible reason for the gap. OpenMP overhead and shared memory bandwidth can also limit scaling, while multi-rank runs add halo exchanges and reductions. We cannot separate these costs from the timing curves, so we do not estimate a measured serial fraction. These tests concern local strong scaling, not multi-node execution.

2.3.4 Profiling Using Performance Counters

We used Intel VTune to profile test_02 with one MPI rank and 1, 2 and 8 OpenMP threads, repeating each analysis three times. Figure 4 summarizes IPC in the computation and estimated L1 load misses.

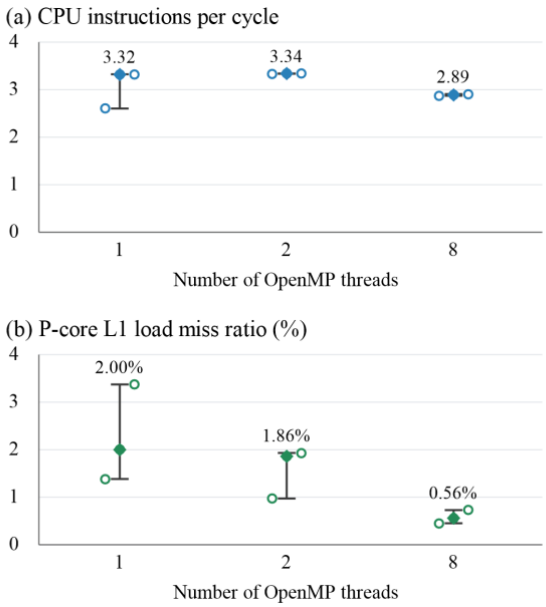


Figure 4. CPU profiling of test_02. Diamonds show medians, whiskers the observed ranges, and dots the three runs. IPC covers the core functions; L1 estimates cover executable-module loads on P-cores.

IPC is similar with one and two threads and lower with eight. The estimated P-core cache miss ratio also falls at eight threads, while the single-thread results show greater variability. These measurements describe different aspects of execution and do not by themselves explain the scaling curves.

3 CUDA Implementation

In the CUDA version, threads update different layer cells. We allocate two layer buffers, storage for the largest particle list, two arrays of partial maxima and a result array. The layer stays on the device between storms. The host uploads each storm, launches the kernels in the default stream and downloads the result pairs. The default block size is 256 threads.

3.1 The Bombardment Kernel

Each thread accumulates one cell in a private float variable. Threads in a block load a tile of particle positions and scaled energies into dynamic shared memory, then reuse it before loading the next tile. For B threads, the tile contains B integers and B floats, requiring 8B bytes. At the default block size of 256, this is 2,048 application bytes.

```
extern __shared__ int positions[];
float *energies = (float *)(positions + blockDim.x);
/* Cooperatively load the current particle tile. */
__syncthreads();
if (active) {
    /* Add tile contributions in particle order. */
}
__syncthreads();
```

Listing 3. Shared tile layout excerpt and the two required barriers.

The first barrier ensures that the tile is ready before any thread reads it. The second ensures that all threads have finished before the tile is overwritten. Threads outside the layer still reach both barriers, but skip the cell update

and final store. An early return would break synchronization. Shared memory reduces repeated particle reads in the code; the reduction in DRAM traffic also depends on how much reuse the global caches already provide.

3.2 The Relaxation Kernel

Relaxation reads the current layer and writes to the other buffer. Neighbouring threads access neighbouring floats, and each interior thread averages the left, centre and right values. The physical endpoints are copied unchanged. Threads beyond N return without accessing memory; this is safe because the kernel has no block-wide barrier. After the kernel, we swap the buffers rather than copy the layer.

The average reads global memory directly, without a shared-memory tile or __restrict__ qualifiers. Boundary checks protect the neighbour accesses. For test_08, the large layer and single particle per cell leave little arithmetic work compared with the amount of layer data that must be read and written.

3.3 The Maximum Detection Phase

Each Block identifies the cells that are strict local maxima with positive energy. It then compares these candidates in shared memory to find the maximum value and its position. This is done using a tree-based reduction, where the number of candidates is halved at each step. As in the CPU implementation, the candidate with the highest energy is selected. If two candidates have the same energy, the one with the smaller position is chosen. Threads without a valid candidate use a neutral value but still participate in synchronization to ensure correct execution.

```
for (int offset = blockDim.x / 2;
     offset > 0; offset >>= 1) {
    if (threadIdx.x < offset)
        shared[threadIdx.x] = better_maximum(
            shared[threadIdx.x],
            shared[threadIdx.x + offset]);
    __syncthreads();
}
```

Listing 4. Shared-memory tree reduction excerpt, with pair storage abbreviated.

Further GPU reduction kernels combine the block results until one pair remains. With N=100 million and B=256, the candidate counts are 390,625, 1,526, 6 and 1. We copy the winning pair into the result array with an 8-byte device-to-device transfer, then download 8 bytes per storm. The reduction needs extra kernel launches, but no global atomics or host-side reduction.

3.4 CUDA Performance Analysis

3.4.1 Speedup Analysis Across Block Sizes

We tested B = 64, 128, 256, 512 and 1024 using separate tuning copies. The only source change allows BLOCK_SIZE to be overridden at compilation; the default source and 256-thread executable are unchanged. All variants keep the same cell mapping and arithmetic order. Changing B also changes the particle tile, shared reduction storage and grid size. As in the CPU tests, each point has one warm-up and five measured runs.

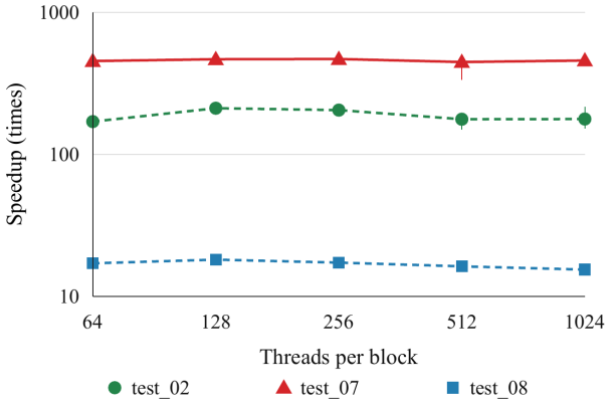


Figure 5. CUDA speedup over the optimized sequential baseline across block sizes. Whiskers use observed CUDA timing ranges with the sequential mean fixed. Timings include allocations, transfers, kernels and frees.

The lowest measured mean times occur at B=128 for test_02 (211.52x), B=256 for test_07 (469.90x) and B=128 for test_08 (18.16x). For test_02, the 0.0002 s gap between B=128 and B=256 is comparable to the reported CUDA standard deviation. Larger blocks do not consistently give better times. They reduce the grid size but also change occupancy and shared-memory use. The many-particle case spends most of its device time in accumulation. With one particle and 100 million cells, initialization and the other layer passes matter more. These choices have the lowest five-run means in this test.

3.5 Hardware Profiling and Throughput Analysis

For each workload and block size, we recorded one CUDA software trace with Nsight Systems, without CPU sampling. We then used Nsight Compute to collect accumulation-kernel counters with multi-pass replay and the default cache and clock controls [3,4]. All profiled outputs match the reference. Speedup calculations use only the uninstrumented runs, since profiling changes execution time.

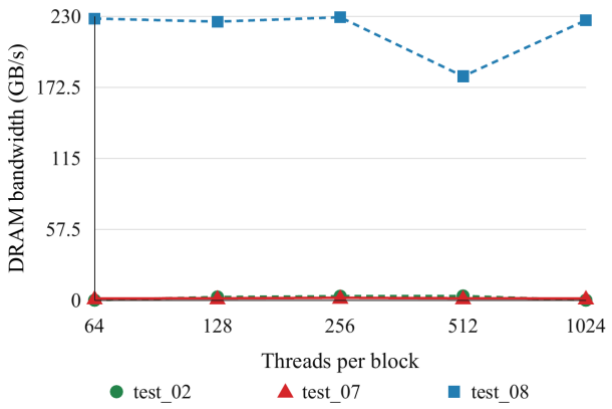


Figure 6. Accumulation-kernel DRAM throughput versus block size, measured with Nsight Compute (dram__bytes.sum.per_second), converted to decimal GB/s. This is device DRAM traffic, not PCIe transfer bandwidth.

At B=256, test_07 accumulation reaches 90.45% SM throughput but only 0.90% DRAM throughput. This points to a compute limit. In contrast, test_08 reaches 89.65% DRAM throughput, or 229.30 GB/s. Its small amount of arithmetic per cell and high DRAM use point to a memory-bandwidth limit within the GPU.

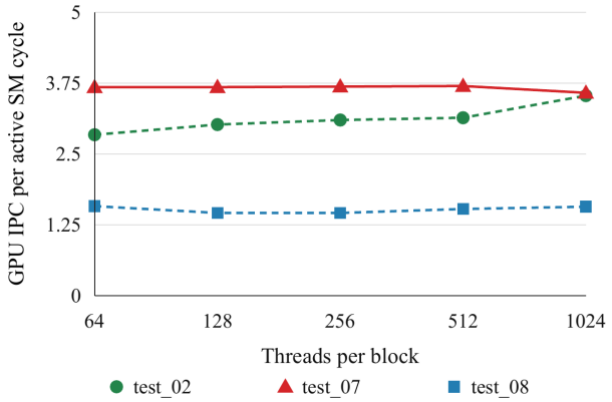


Figure 7. GPU executed IPC per active SM cycle for accumulation, measured with Nsight Compute. This is not CPU instructions per cycle.

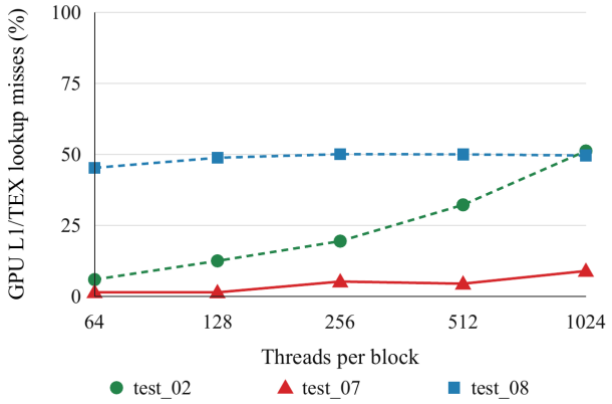


Figure 8. GPU L1/TEX lookup miss fraction, computed as 100 minus Nsight Compute L1/TEX hit rate. The unified GPU cache metric is not a CPU L1 data-cache miss rate.

At a block size of B=256, the GPU achieves an IPC of 3.10 for test_02, 3.69 for test_07, and 1.46 for test_08. The corresponding L1/TEX cache miss rates are 19.47%, 5.24%, and 50.09%.

They are not the fraction of all simulation data that reaches DRAM.

However, IPC and cache miss rates alone are not enough to explain the differences in execution time. A higher IPC does not always result in faster execution, and a low cache miss rate does not necessarily mean that memory bandwidth is not a bottleneck.

Block size also influences GPU occupancy. For example, in test_07, increasing the block size from B=256 to B=1024 reduces achieved occupancy from 98.74% to 66.66%, while the average number of active warps per SM decreases from 47.40 to 32.00. Theoretical occupancy also drops from 100% to 66.67%. These results show that block size affects how effectively GPU resources are utilized and should therefore be considered when comparing execution times.

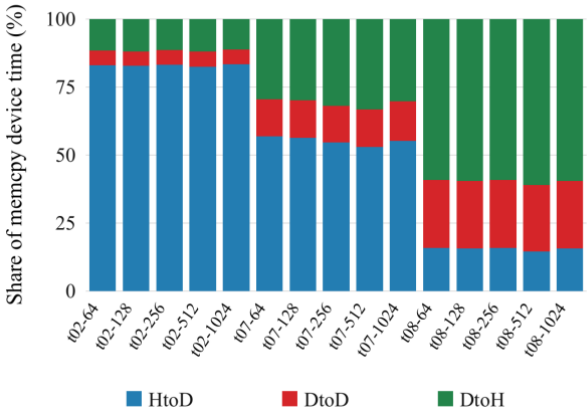


Figure 9. Proportional memcpy device-time breakdown for each workload and block size. Each bar sums to 100% of HtoD, DtoD and DtoH activity only; kernel and memset durations are excluded. One software trace per configuration.

The amount of data transferred from the CPU to the GPU depends heavily on the number of particles: 160,000 bytes for test_02, 40,000 bytes for test_07, and only 8 bytes for test_08. In contrast, both the device-to-device (DtoD) copy and the device-to-host (DtoH) transfer involve only 8 bytes, corresponding to the final result rather than the entire layer, since relaxation uses buffer swapping.

However, a large percentage does not necessarily mean a significant execution cost.

A separate analysis proves that for test_07, the accumulation kernel dominates, accounting for 99.76% of the measured GPU activity. In test_08, the workload is more evenly distributed, with 28.50% spent on accumulation, 27.68% on relaxation, 30.96% on local maxima detection, and the remaining 12.86% on other activities.

These results suggest that the most effective optimization strategy depends on the input. For test_07, which contains many particles, reducing the computational cost of accumulation would be the main priority. For test_08, which contains only one particle

but uses a large layer, reducing the number of passes over the layer and reusing memory allocations could provide greater benefits.

4 Overall Performance

4.1 Comparisons Across Implementations

Table 1 and Figure 10 compare the fastest CPU and CUDA configurations measured in this batch, using the same optimized sequential baseline. Table 2 repeats the B=256 measurements from the tuning sweep.

Test	Seq s	Hybrid s	S CPU	CUDA s	S GPU
test_02	0.9754	0.0857	11.39	0.0046	211.52
test_07	13.0630	0.8195	15.94	0.0278	469.90
test_08	0.5847	0.1653	3.54	0.0322	18.16

Table 1. Mean complete timed regions and speedups for the best observed configurations. Five measured runs per point.

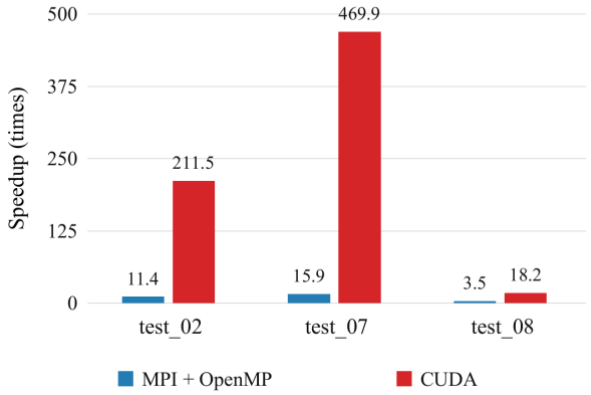


Figure 10. Speedup comparison between the best measured MPI + OpenMP and CUDA configurations on the same host.

4.1.1 Comparative Analysis

Both implementations ran on the same laptop. CUDA benefits from assigning many cells to threads and reusing particle tiles when the accumulation loop is long. The one-particle input gives a smaller speedup because setup and full-layer memory operations take a larger share of the time. MPI + OpenMP distributes the layer across CPU ranks, but these single-host tests do not show how it scales on a cluster.

Test	Hybrid SD	CUDA SD	CUDA B=256
test_02	0.0069	0.0002	0.0048
test_07	0.0266	0.0019	0.0278
test_08	0.0059	0.0005	0.0338

Table 2. Sample standard deviations in seconds for Table 1 and mean time at B=256 from the same tuning sweep.

The runs used normal WSL scheduling, no explicit CPU binding and a laptop GPU. The reported best configurations use the lowest measured means; small differences may change in another batch or on different hardware.

5 Conclusions

This work developed MPI+OpenMP and CUDA implementations of the sequential program, parallelizing cell computations while preserving particle order. The hybrid implementation uses domain decomposition, two halo exchanges and MPI_MAXLOC to find the global maximum. The CUDA implementation uses shared-memory tiles for particle data and a GPU reduction to find the final result. Both implementations also avoid unnecessary data copies during relaxation by swapping buffers.

The experimental results show that performance depends heavily on the input and the parallel implementation used. Absolute speedup is measured against the optimized sequential version, while relative scalability is evaluated using the single-worker version of the same implementation. The CUDA results also show that the best block size depends on the workload. For inputs with many particles, most GPU execution time is spent on accumulation. In contrast, for the large layer with only one particle, execution time is more evenly distributed among accumulation, relaxation and local maxima detection.

6 Future Improvements

These results help identify possible future improvements. For workloads with many particles, optimizing accumulation would likely provide the greatest benefit. For large layers with few particles, reducing memory transfers, avoiding repeated passes over the layer and reusing memory allocations could improve performance. Further experiments could also investigate different block sizes through repeated measurements under controlled conditions.

Finally, the experiments were limited by hardware availability and differences in the testing environments. Collecting profiling data and preparing the performance graphs also presented some difficulties. In addition, multi-node MPI scalability was not evaluated and remains an important area for future testing. Overall, the results show that parallel performance depends not only on the implementation but also on the characteristics of the workload.

A Hardware Specifications and Test Environment

The host has an Intel Core i7-13700H (14 physical cores, 20 logical CPUs) and an NVIDIA GeForce RTX 4060 Laptop GPU (8,188 MiB, compute capability 8.9). Ubuntu 24.04 runs under WSL2 with 20 visible vCPUs and about 7.6 GiB available guest memory. GCC 13.3.0, Open MPI 4.1.6 and CUDA Toolkit 12.6.85 compile the project.

B Compilation Flags and Technical References

B.1 Compilation Flags

We compiled the sequential and hybrid versions with -O3 -ffp-contract=off, adding -fopenmp for the hybrid build. CUDA uses -O3 -arch=sm_89 --fmad=false. We did not enable fast math. Compilation commands specify the flags explicitly because the supplied Makefile defines FLAGS=-O3 without using it in the recipes. The tuning copies add -DBLOCK_SIZE=B.

-O3 enables compiler optimization, -fopenmp enables the OpenMP directives, and sm_89 targets the RTX 4060. We disabled contraction and CUDA fused multiply-add to reduce changes to the ordered float calculation. This does not guarantee bitwise equivalence.

B.2 Dataset Summary and Validation

Workload	N cells	Particles	Waves
test_02	20,000	20,000	1
test_07	1,000,000	5,000	1
test_08	100,000,000	1	1

Table 3. Timed workloads use the first wave of each supplied test family. test_02 uses 20,000 cells.

Validation checks the reported results against the sequential reference, without establishing bitwise equality of every layer element.

Timers exclude input reading, process launch and output. CUDA context setup also occurs before its timer, while allocations, transfers, kernels and frees are included. The MPI main has barriers before and after core() and reports rank 0's interval.

B.3 Technical References

[1] A. Gonzalez-Escribano and E. Rodriguez-Gutiez. Storms of high-energy particles. EduHPC 2018 Peachy Assignment. Supplied handout and sequential source.

[2] MPI Forum. MPI 4.1, MINLOC and MAXLOC. www.mpi-forum.org/docs/mpi-4.1/mpi41-report/node133.htm.

[3] NVIDIA. Nsight Systems User Guide. docs.nvidia.com/nsight-systems/UserGuide/index.html.

[4] NVIDIA. Nsight Compute Profiling Guide. docs.nvidia.com/nsight-compute/ProfilingGuide/index.html.

[5] Cornell University Center for Advanced Computing. Amdahl's Law. Cornell Virtual Workshop. cvw.cac.cornell.edu/parallel/efficiency/amdahls-law.

C Sequential Baseline Timing Analysis

All speedups use the optimized sequential mean from the same host and input. Absolute speedup and efficiency include the changes to the hybrid implementation. Relative scalability instead starts from hybrid single-worker execution, so it measures the gain from adding workers without counting those implementation changes again.